

Create and Running Docker Container

Virtualizzazione e integrazione di sistemi

Vittorio Ghini

a.a. 2024-25

Ingegneria e scienze informatiche

Università di Bologna

Contents

- 0. Getting Info on Docker
- 1. Selecting containers
- 2. Parameters of Docker run command
 - 2.1. Running container interactively
 - 2.2. Running container and a specific command
 - 2.3. Environment variables
 - 2.4. Running container in Background
 - 2.5. Networks
 - 2.6. Default bridge network
 - 2.7. Networking from container point of view
 - 2.8. Published ports
 - 2.9. IP addresses and hostname of container
 - 2.10. DNS services of container
 - 2.11. Container filesystem and storage
 - 2.12. Volumes, bind mounts, tmpfs mounts
 - 2.13. How to mount shared storage (bind mounts) to container
 - 2.14. AutoRemove container on exit

0. Info (1/3)

- Docker has three essential components:
 - Docker Engine
 - Docker Tools
 - Docker Registry
- Docker Engine provides the core capabilities of managing containers. It interfaces with the underlying Linux operating system to expose simple APIs to deal with the lifecycle of containers.
- Docker Tools are a set of command-line tools that talk to the API exposed by the Docker Engine. They are used to run the containers, create new images, configure storage and networks, and perform many more operations that impact the lifecycle of a container.
- Docker Registry is the place where container images are stored. Each image can have multiple versions identified through unique tags. Users pull existing images from the registry and push new images to it.
 - [Docker Hub](#) is a hosted registry managed by [Docker, Inc.](#)
 - It's also possible to run a registry within your own environments to keep the images closer to the engine.

0. Info (2/3)

- The following command shows the details of Docker Engine deployed in the environment

docker info

Containers: 1

Running: 0

Paused: 0

Stopped: 1

Images: 13

Server Version: 18.09.3

Storage Driver: overlay2

Backing Filesystem: extfs

Supports d_type: true

Native Overlay Diff: true

Logging Driver: json-file

Cgroup Driver: cgroupfs

Plugins:

Volume: local

Network: bridge host macvlan null overlay

Log: awslogs fluentd gcplogs gelf journald json-file local
logentries splunk syslog

Swarm: inactive

Runtimes: runc

Default Runtime: runc

Init Binary: docker-init

containerd version: e6b3f5632f50dbc4e9cb6288d911bf4f5e95b18e

runc version: 6635b4f0c6af3810594d2770f662f34ddc15b40d

init version: fec3683

Security Options:

apparmor

seccomp

Profile: default

Kernel Version: 4.15.0-46-generic

Operating System: Ubuntu 18.04.2 LTS

OSType: linux

Architecture: x86_64

CPUs: 1

Total Memory: 1.947GiB

Name: vic

ID: KFSF:FQ7Y:C5J4:Y3GN:WZCO:5OLY:UYMI:2OXT:2MXL:GNWF:FKBE:2X77

Docker Root Dir: /var/lib/docker

Debug Mode (client): false

Debug Mode (server): false

Username: vittorioghini

Registry: <https://index.docker.io/v1/>

Labels:

Experimental: false

Insecure Registries:

127.0.0.0/8

Live Restore Enabled: false

Product License: Community Engine

0. Info (3/3)

- The next command verifies that the Docker Tools are properly installed and configured. It should print the version of both Docker Engine and Tools.

docker version

Client:

Version: 18.09.3

API version: 1.39

Go version: go1.10.8

Git commit: 774a1f4

Built: Thu Feb 28 06:53:11 2019

OS/Arch: linux/amd64

Experimental: false

Server: Docker Engine - Community

Engine:

Version: 18.09.3

API version: 1.39 (minimum version 1.12)

Go version: go1.10.8

Git commit: 774a1f4

Built: Thu Feb 28 05:59:55 2019

OS/Arch: linux/amd64

Experimental: false

1. Selecting container

Filters on docker ps command allow selecting containers.

`$()` is bash command substitution

```
docker ps -a -f status=exited
```

```
docker rm $(docker ps -a -f status=exited -q)
```

The -q flag on docker ps command shows the containers ID only

```
docker ps -a -q
```

```
docker stop $(docker ps -a -q)
```

```
docker rm $(docker ps -a -q)
```

1. Exercise - create Docker image from ubuntu with netcat and netstat

```
apt-cache search netcat                ( search netcat package – netcat )  
apt-cache search netstat              ( search netstat package - net-tools )  
ping <- iputils-ping                  ifconfig <- net-tools                  wget curl
```

```
docker run -it --name ubuntu ubuntu
```

```
apt update
```

```
apt install netcat net-tools iputils-ping wget curl iproute2
```

```
exit
```

```
docker ps -a -q -f status=exited -f "name=ubuntu"
```

```
docker commit -m added_nc_netstat -a "Vic" $(docker ps -a -q -f status=exited -f  
"name=ubuntu") vic/ubuntu_with_nc_netstat
```

or simply

```
docker commit -m added_nc_netstat -a "Vic" ubuntu vic/ubuntu_with_nc_netstat
```

```
docker rm ubuntu
```

2. Parameters of run command

`docker run [OPTIONS] IMAGE [COMMAND] [ARG...]`

- A large set of parameter can be specified when running a Docker container.

<code>-e list</code>	Set environment variables
<code>-d</code>	Run container in background and print container ID
<code>-i</code>	Keep STDIN open even if not attached
<code>--name string</code>	Assign a name to the container
<code>--network string</code>	Connect a container to a network (default "default")
<code>-p list</code>	Publish a container's port(s) to the host
<code>-h string</code>	Container hostname
<code>--ip string</code>	IPv4 address (e.g., 172.30.100.104)
<code>--ip6 string</code>	IPv6 address (e.g., 2001:db8::33)
<code>-v list</code>	Mount a volume
<code>--expose list</code>	Expose a port or a range of ports
<code>--rm</code>	Automatically remove the container when it exits

2.1. Running a Docker container interactively

- **Containers can be interactive.** As an example, let's run a container using the latest image of Ubuntu. The combination of the `-i` and `-t` switches gives you **interactive shell access** (using `stdin/stdout/stderr`) into the container, by running the container in foreground mode:

```
docker run -it ubuntu
```

or

```
docker run -it --name myubuntu ubuntu
```

- where the option `--name myubuntu` assigns the name `myubuntu` to the container. If no option `--name` is provided, docker creates and assign a random name to the container.
- Your command prompt should change to reflect the fact that you're now working inside the container and should take this form:

```
root@d9b100f2f636:/#
```
- Note the container id in the command prompt. In this example, it is `d9b100f2f636`. You'll need that container ID later to identify the container when you want to remove it.

2.1.1. Running bash commands interactively inside a container

- **Now you can run any command inside the myubuntu container.** For example, let's update the package database inside the container. You don't need to prefix any command with sudo, because you're operating inside the container as the root user:
- Check if the package node.js already exist in your system, but it does not exist

```
node -v
```

```
bash: node: command not found
```
- Install the node.js package.

```
apt update
```
- Then install any application in it. Let's install Node.js:

```
apt install nodejs
```
- This installs Node.js in the container from the official Ubuntu repository. When the installation finishes, verify that Node.js is installed:

```
node -v
```
- You'll see the version number displayed in your terminal: `Outputv8.10.0`
- Any changes you make inside the container only apply to that container.
- **To exit the container, type exit at the prompt. The shell terminates, the container stops.**
- If you restart the container, the previous node.js installation has been lost.
- Try again `"docker run -it ubuntu"` and then run `node -v`

2.2. Running a container and a specific command

generic docker run syntax:

```
docker run [OPTIONS] IMAGE [COMMAND] [ARG...]
```

-d	Run container in background and print container ID
-i	Keep STDIN open even if not attached
--name string	Assign a name to the container
--rm	Automatically remove the container when it exits
.....	

example:

```
docker run -it --name myubuntu ubuntu /usr/bin/find / -iname '*.sh'
```

when the command *find* terminates, the container stops.

2.3. Environment variables

`-e "VARNAME=VARCONTENT"` Set environment variables

```
docker run -it -e "PIPPO=PIPPOEPLUTO" --name myubuntu ubuntu
```

```
root@bd78a453e0c7:/# echo $PIPPO
```

```
PIPPOEPLUTO
```

```
root@bd78a453e0c7:/# bash
```

```
root@bd78a453e0c7:/# ps
```

PID	TTY	TIME	CMD
-----	-----	------	-----

1	pts/0	00:00:00	bash
---	-------	----------	------

10	pts/0	00:00:00	bash
----	-------	----------	------

15	pts/0	00:00:00	ps
----	-------	----------	----

```
root@bd78a453e0c7:/# echo $PIPPO
```

```
PIPPOEPLUTO
```

```
root@bd78a453e0c7:/# env | grep PIPPO
```

```
PIPPO=PIPPOEPLUTO
```

```
root@bd78a453e0c7:/# exit
```

```
exit
```

2.4. Running in Background

-d Run container in background and print container ID

-i Keep STDIN open even if not attached

```
docker run -d --name myubuntu ubuntu
```

```
docker ps -a
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
37b65ce92fe4	ubuntu	"/bin/bash"	4 seconds ago	Exited (0) 3 seconds ago		myubuntu

Executes /bin/bash with no stdin, thus it terminates.

```
docker run -d -i --name myubuntu1 ubuntu
```

```
docker ps -a
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
fe55f2fb05c	ubuntu	"/bin/bash"	3 seconds ago	Up 1 second		myubuntu1

```
docker kill myubuntu1
```

```
docker ps -a
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
fe55f2fb05c	ubuntu	"/bin/bash"	47 seconds ago	Exited (137) 1 second ago		myubuntu1

```
docker rm myubuntu myubuntu1
```

2.4.1. Execute interactively a command in a running background container

```
$ docker run -d -it --name myubuntu1 ubuntu
```

```
$ docker ps -a
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
fe55f2fb05c	ubuntu	"/bin/bash"	3 seconds ago	Up 1 second		myubuntu1

docker exec [OPTIONS] CONTAINER COMMAND [ARGS]

-d, --detach	Detached mode: run command in the background
-i, --interactive	Keep STDIN open even if not attached
-t, --tty	Allocate a pseudo-TTY
-u, --user string	Username or UID (format: <name uid>[:<group gid>])
-w, --workdir string	Working directory inside the container

```
$ docker exec -it -u root:root -w /usr/local/ myubuntu find ./ -iname '*ma*'
```

```
./man
```

```
./share/man
```

```
$ docker ps -a
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
183c45e5d2de	ubuntu	"/bin/bash"	10 minutes ago	Up 10 minutes		myubuntu

2.4.2. Execute a background command in a running background container

```
$ docker run -d -it --name myubuntu1 ubuntu
```

```
$ docker ps -a
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
fe555f2fb05c	ubuntu	"/bin/bash"	3 seconds ago	Up 1 second		myubuntu1

```
$ docker exec -d -u root:root -w /usr/local/ myubuntu find ./ -iname '*ma*'
```

```
$ docker ps -a
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
183c45e5d2de	ubuntu	"/bin/bash"	10 minutes ago	Up 10 minutes		myubuntu

2.4.3. Attach an interactive shell to a running background container

```
$ docker run -d -it --name myubuntu1 ubuntu
```

```
$ docker ps -a
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS
fe555f2fb05c	ubuntu	"/bin/bash"	3 seconds ago	Up 1 second	
myubuntu1					

generic syntax

```
docker attach containerName or containerId
```

example:

```
docker attach myubuntu
```

- after this, the stdin/stdout/stderr streams are connected to the stdin/stdout/stderr streams of a bash running in the container.
- If you executes the exit command in the shell the container stops.

2.4.4. Detach a foreground container and turn it stopped but not exited

```
$ docker run -it --name myubuntu1 ubuntu
```

From the prompt of the container, you can type commands.

To detach the container you must type CTRL+p and CTRL+q

This detach your shell from the container.

The container stops but does not exit.

You can now attach again to the container

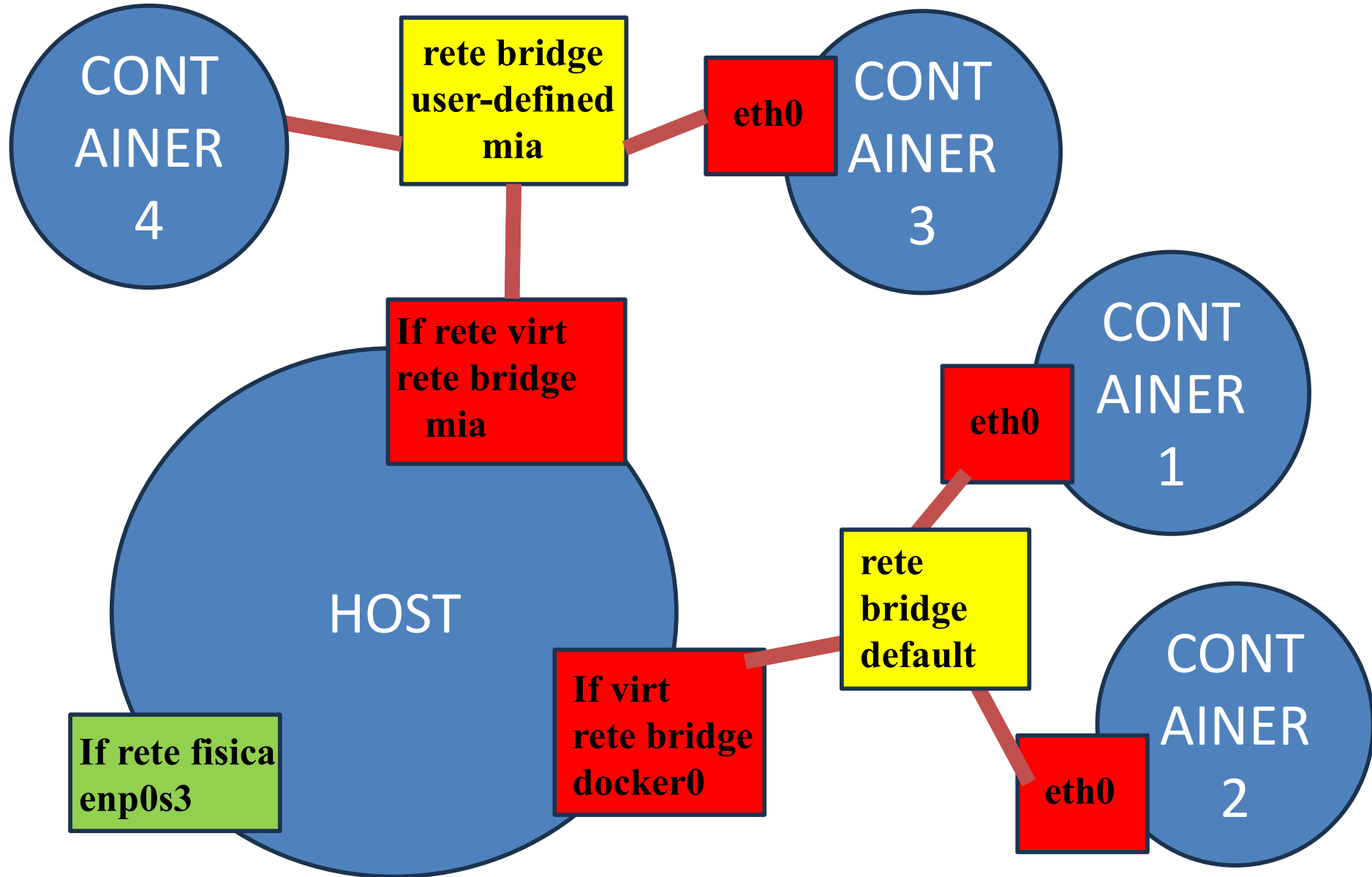
```
$ docker attach ubuntu
```

2.5. Networks (1/5)

- Docker's networking subsystem is pluggable, using drivers.
- Several drivers exist by default, and provide core networking functionality:
bridge, host, overlay, macvlan, none, custom network plugins.
- You can assign a driver to a container using the **--network** option.
- If no specified, the default is the **bridge** network driver.
- In fact, **when you start Docker, a default bridge network (also called bridge) is created automatically, and newly-started containers connect to it unless otherwise specified. The default bridge networks allow container to connect to external service.**
- You can also create user-defined custom bridge networks.
- **A bridge network allows containers connected to the same bridge network to communicate, while providing isolation from containers which are not connected to that bridge network.**
- The Docker bridge driver automatically installs rules in the host machine so that containers on different bridge networks cannot communicate directly with each other.
- Bridge networks apply to containers running on the **same** Docker daemon host. For communication among containers running on different Docker daemon hosts, you can either manage routing at the OS level, or you can use an overlay network.

2.5. Networks (2/5)

- Each user-defined bridge network provides a virtual network interface to the host.



2.5. Networks (3/5)

- **bridge**: If you don't specify a driver, this is the type of network you are creating. Bridge networks are usually used when your applications run in standalone containers that need to communicate to each other. Bridge networks allow container to connect to external service. You can create a custom bridge.
- **host**: For standalone containers, **remove network isolation between the container and the Docker host, and use the host's networking directly.** host is only available for swarm services on Docker 17.06 and higher.
- **overlay**: Overlay networks connect multiple Docker daemons together and enable swarm services to communicate with each other. You can also use overlay networks to facilitate communication between a swarm service and a standalone container, or between two standalone containers on different Docker daemons. This strategy removes the need to do OS-level routing between these containers.
- **macvlan**: Macvlan networks allow you to assign a MAC address to a container, making it appear as a physical device on your network. The Docker daemon routes traffic to containers by their MAC addresses. Using the macvlan driver is sometimes the best choice when dealing with legacy applications that expect to be directly connected to the physical network, rather than routed through the Docker host's network stack.
- **none**: For this container, disable all networking. Usually used in conjunction with a custom network driver.

2.5. Networks (4/5)

example 1: host

ifconfig

enp0s3: inet 10.0.2.15

docker0: inet 172.17.0.1

nc -l -p 60000

parole parole parole

Same host, another shell

Container uses default bridge network

docker run -it --name nc vic/ubuntu_with_nc_netstat

ifconfig

eth0: inet 172.17.0.2

root@b6abc01c64b1:/# nc 10.0.2.15 60000

parole parole parole

example 2: host

ifconfig

enp0s3: inet 10.0.2.15

docker0: inet 172.17.0.1

nc -l -p 60000

Same host, another shell

Container uses none network

docker run -it --network none --name nc

vic/ubuntu_with_nc_netstat

ifconfig

lo: inet 127.0.0.1

root@b6abc01c64b1:/# nc 10.0.2.15 60000

(UNKNOWN) [10.0.2.15] 60000 (?) : **Network is unreachable**

2.5. Networks (5/5)

example 3: host

ifconfig

enp0s3: inet 10.0.2.15

docker0: inet 172.17.0.1

vic@vic:~\$ nc localhost
61000

ffdlfcdx

Same host, another shell

Container uses host network

- `docker run -it --network host--name nc
vic/ubuntu_with_nc_netstat`

ifconfig

enp0s3: inet 10.0.2.15

docker0: inet 172.17.0.1

lo: inet 127.0.0.1

root@vic:/# nc -l -p 61000

ffdlfcdx

^C

Try to use "nc www.cs.unibo.it 80" and digit "ciao"

2.6. Default bridge network

- **Enable forwarding from Docker containers to the outside world**

By default, traffic from containers connected to the default bridge network is **not** forwarded to the outside world. To enable forwarding, you need to change two kernel-level settings. In fact, these are not Docker commands and they affect the Docker host's kernel.

- 1 Configure the Linux kernel to allow IP forwarding.

```
sysctl net.ipv4.conf.all.forwarding=1
```

- 2 Change the policy for the iptables FORWARD policy from DROP to ACCEPT.

```
sudo iptables -P FORWARD ACCEPT
```

These settings do not persist across a reboot, so you may need to add them to a start-up script.

- **Use the default bridge network**

The default bridge network is considered a legacy detail of Docker and is not recommended for production use. User-defined bridge network should be used.

- **Connect a container to the default bridge network**

If you do not specify a network using the `--network` flag, your container is connected to the default bridge network by default.

2.6. Configure default bridge network

- **Enable forwarding from Docker containers to the outside world**

To configure the default bridge network, you may specify options in `daemon.json`. Here is an example `daemon.json` with several options specified. Only specify the settings you need to customize

```
{  
    "bip": "192.168.1.5/24",  
    "fixed-cidr": "192.168.1.5/25",  
    "fixed-cidr-v6": "2001:db8::/64",  
    "mtu": 1500,  
    "default-gateway": "10.20.1.1",  
    "default-gateway-v6": "2001:db8:abcd::89",  
    "dns": ["10.20.1.2", "10.20.1.3"]  
}
```

Restart Docker for the changes to take effect.

2.7. Networking from container point of view (1/2)

The type of network a container uses, whether it is a bridge, an overlay, a macvlan network, or a custom network plugin, is transparent from within the container. Assuming the container is not using the none network driver, from the container's point of view, it has a network interface with

- an IP address,
- a gateway,
- a routing table,
- a DNS services,
- and other networking details.

Published ports

By default, when you create a container, it does not publish any of its ports to the outside world.

To make a port available to services outside of Docker, or to Docker containers which are not connected to the container's network, use the `--publish` or the `-p` flag. This creates a firewall rule which maps a container port to a port on the Docker host.

2.7. Networking from container point of view (2/2)

Here are some examples.

This topic is about networking concerns from the point of view of the container.

To configure the default bridge network, you may specify options in daemon.json.

Here is an example daemon.json with several options specified. Only specify the settings you need to customize

```
{  
    "bip": "192.168.1.5/24",  
    "fixed-cidr": "192.168.1.5/25",  
    "fixed-cidr-v6": "2001:db8::/64",  
    "mtu": 1500,  
    "default-gateway": "10.20.1.1",  
    "default-gateway-v6": "2001:db8:abcd::89",  
    "dns": ["10.20.1.2", "10.20.1.3"]  
}
```

2.8. Published ports (1/4)

By default, when you create a container, it does not publish any of its ports to the outside world.

To make a port available to services outside of Docker, or to Docker containers which are not connected to the container's network, use the `--publish` or the `-p` flag. This creates a firewall rule which maps a container port to a port on the Docker host.

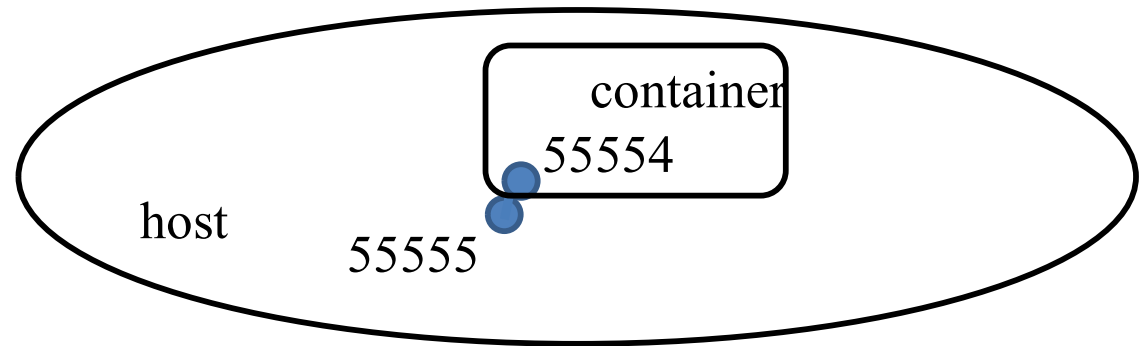
Flag value	Description
<code>-p 8080:80</code>	Map TCP port 80 in the container to port 8080 on the Docker host.
<code>-p 192.168.1.100:8080:80</code>	Map TCP port 80 in the container to port 8080 on the Docker host for connections to IP 192.168.1.100 of the host
<code>-p 8080:80/udp</code>	Map UDP port 80 in the container to port 8080 on the Docker host.
<code>-p 8080:80/tcp -p 8080:80/udp</code>	Map TCP port 80 in the container to TCP port 8080 on the Docker host, and map UDP port 80 in the container to UDP port 8080 on the Docker host.

2.8. Published port (2/4)

Publish a container port **available to the host** and **available to the other containers**

-p HostPort:ContainerPort

-p 55555:55554



Requires `--network` parameter allows using network

example: two netcat program,
one in the host and the second
in the container

To see all the port defined in a container, run

`docker port ContainerId`

2.8. Published port (3/4)

Create **one** Container: run ubuntu with netcat

```
docker run -it -p 55555:55554 --name nc vic/ubuntu_with_nc_netstat
```

```
root@398b3ce9657d:/# nc -l -p 55554
```

stringa scritta come input

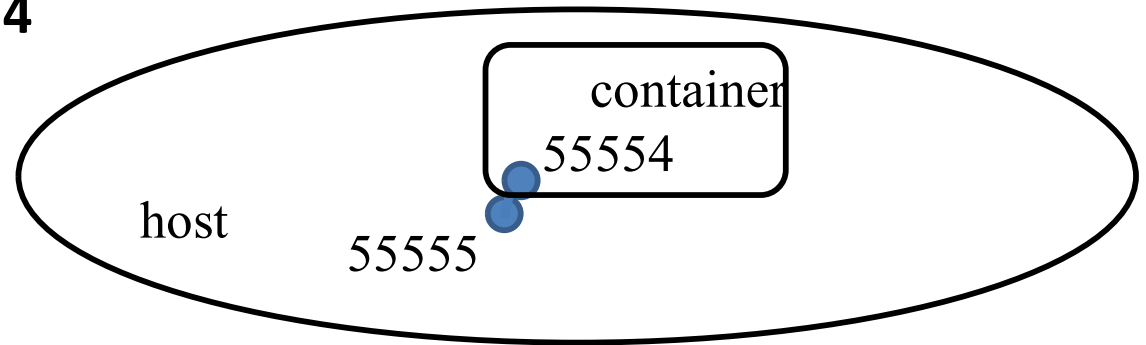
^Z

```
[1]+ Stopped nc -l -p 55554
```

```
root@398b3ce9657d:/# netstat
```

Active Internet connections (w/o servers)

Proto	Recv-Q	Send-Q	Local Address	Foreign Address	State
tcp	0	0	398b3ce9657d:55554	172.17.0.1:42854	ESTABLISHED



Host side:

```
vic:# nc localhost 55555
```

stringa scritta come input

^C

```
docker port 398b3ce9657d
```

```
55554/tcp -> 0.0.0.0:5555
```

2.8. Published port (4/4)

Create **two** Container: run ubuntu with netcat

Container 1

```
docker run -it -p 55555:55554 --name nc vic/ubuntu_with_nc_netstat
```

```
root@398b3ce9657d:/# nc -l -p 55554
```

stringa scritta come input

^Z

```
[1]+ Stopped nc -l -p 55554
```

```
root@398b3ce9657d:/# netstat
```

Active Internet connections (w/o servers)

Proto	Recv-Q	Send-Q	Local Address	Foreign Address	State
tcp	0	0	398b3ce9657d:55554	172.17.0.1:43022	ESTABLISHED

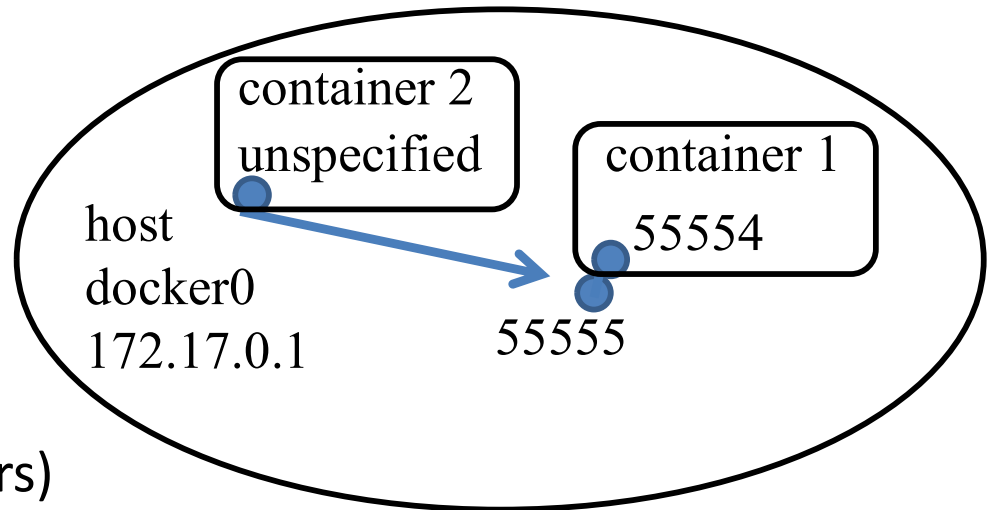
Container 2

```
docker run -it --name nc1 vic/ubuntu_with_nc_netstat
```

```
vic:# nc 172.17.0.1 55555
```

stringa scritta come input

^C



2.8. Published port (5/4)

ATTENZIONE, ANTICIPAZIONE: per consentire ad un container A di connettersi alla porta portaB di un container B **NON E' NECESSARIO ESPORRE SULL'HOST la porta portaB del container B** mediante l'opzione `-p portaB:portaB` di `docker run`, **ANZI E' FORTISSIMAMENTE SCONSIGLIATO.**

Vedremo che **il modo giusto di consentire al container A di connettersi al container B è quello di creare una rete virtuale di tipo bridge personalizzata su cui entrambi i container hanno una interfaccia di rete virtuale.**

In tal modo il container A potrà connettersi al container B senza dover conoscere l'indirizzo del container B bensì solo conoscendo il nome del container B, grazie al DNS embedded di docker.

Al posto di una rete di tipo bridge personalizzata si potrebbe usare la rete bridge di default, in tal caso però avremmo un problema di sicurezza e una limitazione.

Il problema di sicurezza è che tutti i container sulla rete bridge di default vedrebbero la porta del container B.

La limitazione è che sulla rete bridge di default non opera il DNS embedded di docker, quindi il container A dovrebbe conoscere l'indirizzo IP del container B, non basta che ne conosca il nome.

Quindi è bene usare una rete bridge personalizzata, non quella bridge di default. 31

2.9. IP addresses and hostname of container

- By default, the container is assigned an IP address for every Docker network it connects to. The IP address is assigned from the pool assigned to the network, so the Docker daemon effectively acts as a DHCP server for each container. Each network also has a default subnet mask and gateway.
- When the container starts, it can only be connected to a single network, using the `--network` option.
 - When you start a container using the `--network` flag, you can specify the IP address assigned to the container on that network using the `--ip` or `--ip6` flags.
 - In the same way, a container's hostname by default assumes the values of the container's ID in Docker. You can override the hostname using `--hostname`.
- However, you can connect a running container to multiple networks using the command *docker network connect*.
 - When you connect an existing container to a different network using `docker network connect`, you can use the `--ip` or `--ip6` flags on that command to specify the container's IP address on the additional network.
 - When connecting to an existing network using `docker network connect`, you can use the `--alias` flag to specify an additional network alias for the container on that network.

2.10. DNS services of container

- By default, a container inherits the DNS settings of the Docker daemon, including the `/etc/hosts` and `/etc/resolv.conf`.
- You can override these settings on a per-container basis when creating a container.

Flag value	Description
<code>--dns</code>	The IP address of a DNS server. To specify multiple DNS servers, use multiple <code>--dns</code> flags. If the container cannot reach any of the IP addresses you specify, Google's public DNS server 8.8.8.8 is added, so that your container can resolve internet domains.
<code>--dns-search</code>	A DNS search domain to search non-fully-qualified hostnames. To specify multiple DNS search prefixes, use multiple <code>--dns-search</code> flags.
<code>--dns-opt</code>	A key-value pair representing a DNS option and its value. See your operating system's documentation for <code>resolv.conf</code> for valid options.
<code>--hostname</code>	The hostname a container uses for itself. Defaults to the container's ID if not specified.

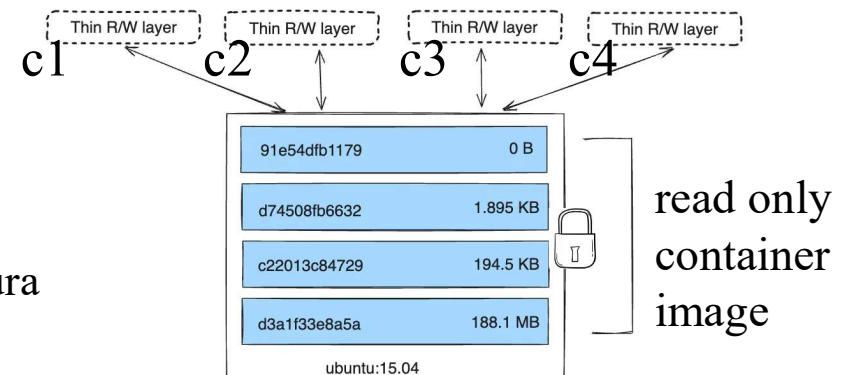
2.11.0 docker images and filesystems

- MOLTO BELLO: vedere parte iniziale su immagini e filesystem
- <https://www.linkedin.com/pulse/docker-container-platform-images-e-repository-3-alessio-mario/>

2.11.0 Il filesystem dei container e gli storage

- Le immagini dei container sono salvati nel registry del daemon docker, cioè nel filesystem dell'host in una directory predefinita del daemon docker all'interno della cosiddetta docker area, la directory `/var/lib/docker/`. Questa docker area può anche essere una partizione separata, avente un filesystem specializzato (ad esempio btrfs) per rendere efficiente la memorizzazione delle immagini dei container e delle loro modifiche, e montata in `/var/lib/docker` (<https://docs.docker.com/storage/storagedriver/btrfs-driver/>).
- Quando vengono creati, i container possiedono un filesystem iniziale che è quello contenuto nella loro immagine. Durante l'esecuzione, il container può modificare il contenuto del proprio filesystem. Queste modifiche vengono salvate "temporaneamente" sul filesystem dell'host, nella docker area, in uno strato differenziale "al top" rispetto all'immagine originale del container. Si parla di filesystem di tipo Copy-on-Write (CoW). I container che eseguono a partire da una stessa immagine condividono in sola lettura il filesystem dell'immagine nella docker area. Quando un container deve modificare il proprio filesystem ne crea una copia che mantiene SOLO le differenze rispetto all'originale. Ciò è efficiente perché esisterà solo una istanza (in sola lettura) dell'immagine originale e poi ogni container avrà un proprio copia con le sole modifiche, risparmiando spazio disco.

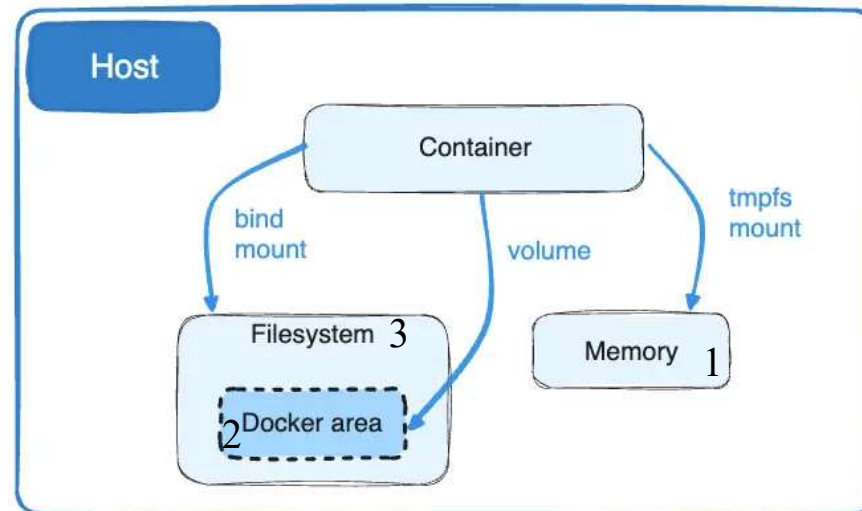
I container che nascono da una stessa immagine condividono in sola lettura l'immagine originale (cioè il filesystem originale) ma ciascuno aggiunge un proprio livello in lettura/scrittura in cui mantiene le differenze rispetto all'originale.



- Il container avrebbe a disposizione tutto lo spazio disco dell'host ma il daemon docker definisce una dimensione massima occupabile dal filesystem di ciascun container pari a 10 G. Questa impostazione può essere modificata configurando il daemon docker (`/etc/docker/daemon.json`).
- Nel momento della **terminazione** del container le modifiche vengono congelate e nel momento della **rimozione** del container queste modifiche vengono perse.

2.12.0 volumes, bind mounts, tmpfs mounts

- Alcune risorse di tipo storage dell'host possono essere messe a disposizione di un container che vedrà queste risorse come se fossero partizioni montate nel proprio filesystem.
<https://docs.docker.com/storage/volumes/>
- Esistono tre tipi di storage esterni, denominati genericamente volumi, e sono:
 1. **Tmpfs Mounts** - partizioni temporanee in memoria.
 2. **Bind Mounts** - partizioni collegate a directory dell'host.
 3. **Managed Volumes, named e unnamed** - Volumi gestiti [da docker] persistenti con o senza nome.



2.12.0 Motivazioni dell'aggiunta di Volumi al filesystem dei container

1. Se il container necessita di salvare delle modifiche al filesystem affinché **sopravvivano alla propria rimozione** oppure affinché **siano condivise con altri container**, allora deve collocare queste modifiche in delle partizioni (**storage esterni al container**) montate in punti del filesystem del container. Le modifiche apportate a queste partizioni **non vengono salvate incrementalmente nel filesystem principale del container** (cioè non nella directory predefinita dell'host) bensì, ovviamente, negli storage esterni.

Alla terminazione del container, gli storages esterni sopravvivono e mantengono le modifiche.

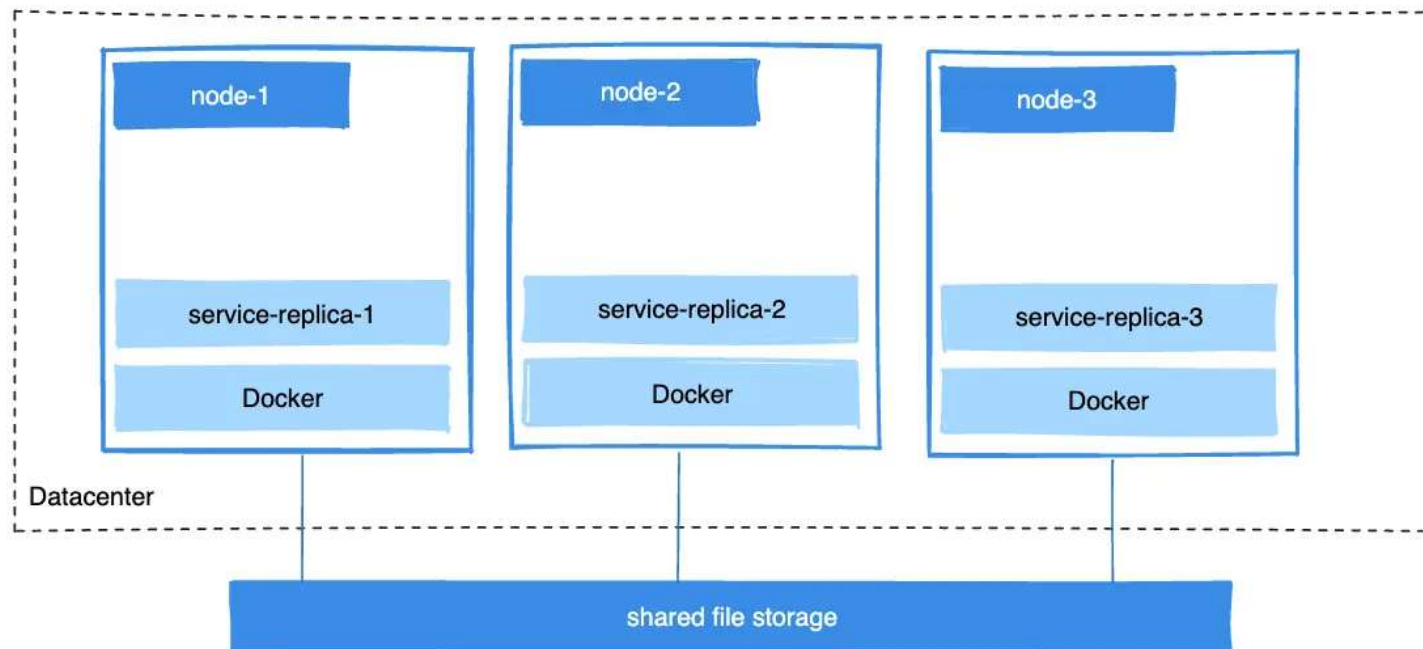
Gli storage esterni possono essere acceduti contemporaneamente da più container, realizzando una condivisione di questi spazi disco.

Lo spazio occupato negli storage esterni non concorre a saturare il limite di spazio disco di 10GB di ciascun container.

2. Esiste un altro motivo per aggiungere dei volumi. I container che eseguono a partire da una stessa immagine condividono in sola lettura il filesystem dell'immagine nella docker area. Quando un container deve modificare il proprio filesystem ne crea una copia che mantiene SOLO le differenze rispetto all'originale. Ciò è efficiente perché esisterà solo una istanza (in sola lettura) dell'immagine originale e poi ogni container avrà un proprio copia con le sole modifiche, risparmiando spazio disco. Però in tal modo le scritture sui files diventano più onerose. **In caso di container che devono modificare spesso i propri files (ad esempio i database) è meglio utilizzare files in storage esterni che hanno filesystem che NON usano il modello Copy-on-Write.**
3. Inoltre, se il container sa che le **modifiche apportate al filesystem dovranno essere eliminate** dopo la propria terminazione, è meglio risparmiare scritture su disco montando un volume esterno di tipo Tmpfs mount che NON scrive su disco bensì in memoria. Si risparmia l'accesso a disco.

2.12.0 Motivazioni dell'aggiunta di Volumi al filesystem dei container che lavorano su machine diverse

4. **Condividere dati tra macchine.** Può essere necessario, per fault tolerance (resilienza) configurare più repliche di uno stesso servizio che devono perciò accedere agli stessi file. Un modo è creare un volume che consente al container di scrivere normalmente dei file. Ma il volume sfrutta un driver che va a leggere e scrivere quei file su uno storage remoto, senza che il container debba specificare nulla. Il driver del volume consente di astrarre dalla tecnologia dello storage. Esempi di driver: NFS, sshfs.



2.13.1. Bind Mounts (1/4)

I Containers sono effimeri, ciò significa che ogni cosa salvata dentro al container sarà persa quando il container verrà terminato e rimosso. Per rendere persistenti dei dati, superando la rimozione del container, dobbiamo attaccare un volume al container.

Bind Mounts sono directories del file system dell'host [o di un host remoto].

Nel comando docker run, l'opzione `-v` può dare a un container o un volume (named o unnamed) o un bind mount o un tmpfs mount a seconda di cosa specifichi.

Per dare al container un bind mount, l'**opzione `-v`** del comando docker run **specifica il percorso della directory dell'host che deve essere condivisa e il percorso nel filesystem del container in cui il volume sarà montato.**

`-v HostDirectory:ContainerDirectory`

Il seguente esempio

- crea un container che esegue un http web server in attesa sulla porta 80 del container,
- Rende disponibile il server sulla porta 8888 dell'host,
- E salva le pagine html in un volume esterno al container.
- Le pagine quindi sono persistenti.

2.13.1. Bind Mounts (2/4)

example of shared storage

- Create host directory in `/home/vic/htdocs`

```
mkdir /home/vic/htdocs
```

- Run Apache web server in background.

The -v switch points the htdocs directory within the container to the host's file system. Any changes made to this directory will be visible at both the locations.

```
docker run -p 8888:80 --name web -d \
```

```
-v /home/vic/htdocs:/usr/local/apache2/htdocs httpd
```

- Access the directory from the container by running the following command that attaches our terminal to the shell of the containers in an interactive mode :

```
docker exec -it web /bin/bash
```

- Navigate to the htdocs folder and create a simple HTML file. Finally, exit the shell to return to the host.

```
cd /usr/local/apache2/htdocs
```

```
echo '<h1>Hello World from Container</h1>' > index.html
```

```
exit
```


2.13.1. Bind Mounts (3/4)

example of shared storage

- Get the web server home page
`curl localhost:8888`
- You can modify the files on the host side while the httpd daemon is running.
- But, you needs to change file permissions, because index.html has been created by the root user.

```
chmod 666 index.html
```

At the host side:

```
echo '<h1>Hello Pippo from Pluto</h1>' > /home/vic/htdocs/index.htm
```

- Get the web server home page again

```
curl localhost:8888
```

Terminates the httpd container directly without stopping (the -f flag forces)

```
docker rm -f web
```

Or, if you preferes, two commands.

```
docker kill web
```

```
docker rm web
```

- You can modify the files on the container side also.

2.13.1. Bind Mounts (4/4)

example of shared storage

- To avoid root is the owner of index.html, you can create the index.html file initially at the host side.

```
touch /home/vic/htdocs/index.html
```

```
chmod 666 /home/vic/htdocs/index.html
```

- Restart the httpd apache web server

```
docker run -p 8888:80 --name web -d \
```

```
    -v /home/vic/htdocs:/usr/local/apache2/htdocs httpd
```

- Get the web server home page again

```
curl localhost:8888
```

2.13.1. Bind Mounts (5/5)

example of shared storage

- Esiste la possibilità di usare una sintassi diversa, per creare un bind mount, così:

```
docker volume create --name my_test_volume --opt type=nfs --opt  
device=/home/jinna/Jinna_Balu/Test_volume --opt o=bind
```

2.14. AutoRemove container on exit

- The `--rm` flag automatically removes the container when it exits.
- Inoltre, con il flag `--rm` vengono automaticamente rimossi anche i volume unnamed.

```
vic@vic:~/$ docker run -it --rm --name nc1 vic/ubuntu_with_nc_netstat
```

```
root@8110c268ff4f:/# exit
```

```
vic@vic:~/$ docker ps -a
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
--------------	-------	---------	---------	--------	-------	-------

No container exists, no unnamed volume exists.